

Phase 2 Database Schema

Overview

This document details the new database models required for Phase 2 features: Notifications, Element Notes, Wishlist Context, and Admin Analytics. All schemas use Prisma ORM with PostgreSQL.

Existing Models (Phase 1 Reference)

User

```
model User {
  id          String      @id @default(cuid())
  email       String      @unique
  firstName   String?
  lastName    String?
  shopifyCustomerId String? @unique
  isStaff     Boolean      @default(false)
  preferredFormat String? // '10mm', '25.4mm', '50mm', 'lucite_cube', 'ampoule'

  createdAt    DateTime @default(now())
  updatedAt    DateTime @updatedAt

  collectionItems CollectionItem[]
  goals          Goal[]
  milestones     Milestone[]

  // NEW RELATIONS (Phase 2)
  elementNotes    ElementNote[]
  notifications    NotificationQueue[]
  notificationPrefs UserNotificationPreferences?
}
```

CollectionItem

```

model CollectionItem {
  id String @id @default(cuid())
  userId String
  elementSymbol String
  state CollectionItemState // 'owned' | 'wishlist' | 'watchlist' | 'missing'
  format String? // '10mm', '25.4mm', '50mm', etc.

  // NEW FIELDS (Phase 2 - Wishlist Context)
  contextLabel WishlistContext?
  contextOverridden Boolean @default(false)
  contextReason String?

  priority Int? @default(0)
  shopifyProductId String?
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  user User @relation(fields: [userId], references: [id])
  element Element @relation(fields: [elementSymbol],
  references: [symbol])
  elementNote ElementNote? // NEW RELATION (Phase 2)

  @@unique([userId, elementSymbol, format])
  @@index([userId, state])
  @@index([elementSymbol])
}

enum CollectionItemState {
  OWNED
  WISHLIST
  WATCHLIST
  MISSING
}

// NEW ENUM (Phase 2)
enum WishlistContext {
  CORE
  EXPLORATION
  ASPIRATIONAL
  UPGRADE_TARGET
}

```

Element

```
model Element   
  id String @id @default(cuid())  
  symbol String @unique  
  name String  
  atomicNumber Int @unique  
  category String  
  group Int?  
  period Int  
  
  collectionItems CollectionItem   
  
  @@index([category])  

```

New Models (Phase 2)

1. NotificationQueue

Purpose: Queue and track all notifications sent to users

```

model NotificationQueue {
  id                String                @id @default(cuid())
  userId            String
  type              NotificationType
  status            NotificationStatus    @default(PENDING)

  // Related entities
  relatedElementId  String?
  relatedProductId  String?
  relatedMilestone  String?

  // Metadata (JSON for flexibility)
  metadata          Json?

  // Relevance and prioritization
  relevanceScore     Int?

  // Delivery tracking
  deliveryMethod     String[]             // ['in_app', 'email']
  scheduledFor       DateTime?
  sentAt             DateTime?
  readAt             DateTime?
  clickedAt          DateTime?

  // Error handling
  errorMessage       String?
  retryCount         Int                  @default(0)

  createdAt          DateTime             @default(now())
  updatedAt          DateTime             @updatedAt

  user               User                 @relation(fields: [userId], references: [id])

  @@index([userId, status])
  @@index([status, scheduledFor])
  @@index([type, createdAt])
}

enum NotificationType {
  COMPLETION_UNLOCK // 1 element away from milestone
  NEAR_COMPLETION   // 90%+ in a category
  WISHLIST_RESTOCK  // Wishlisted item back in stock
  HIGH_RELEVANCE_ARRIVAL // New product matching interests
  MILESTONE_CELEBRATION // Collection milestone reached
}

enum NotificationStatus {
  PENDING // Queued, not yet sent
  SENT    // Successfully delivered
  FAILED  // Delivery failed
  CANCELLED // Cancelled before sending
  READ    // User read the notification (in-app only)
}

```

Key Features:

- `metadata` JSON field stores type-specific data (element name, milestone %, etc.)
- `deliveryMethod` array allows multi-channel delivery
- `relevanceScore` enables prioritization
- Retry logic with `retryCount` and `errorMessage`

Example metadata:

```
{
  "elementSymbol": "Ga",
  "elementName": "Gallium",
  "format": "25.4mm",
  "price": 49.99,
  "currentCompletion": 74.6,
  "nextMilestone": 75,
  "categoryName": "Post-Transition Metals",
  "ownedCount": 88,
  "missingElements": ["Se", "Te", "Po"]
}
```

2. UserNotificationPreferences

Purpose: Store user's notification preferences (opt-in/out per type, frequency limits)

```
model UserNotificationPreferences {
  id String @id @default(cuid())
  userId String @unique

  // Email preferences (per notification type)
  emailCompletionUnlock Boolean @default(true)
  emailNearCompletion Boolean @default(true)
  emailWishlistRestock Boolean @default(true)
  emailHighRelevance Boolean @default(true)
  emailMilestone Boolean @default(true)

  // In-app preferences (per notification type)
  inAppCompletionUnlock Boolean @default(true)
  inAppNearCompletion Boolean @default(true)
  inAppWishlistRestock Boolean @default(true)
  inAppHighRelevance Boolean @default(true)
  inAppMilestone Boolean @default(true)

  // General settings
  emailFrequencyLimit Int @default(3) // Max emails per week
  mutedUntil DateTime? // Temporarily mute all

  // Metadata
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  user User @relation(fields: [userId], references: [id])
}
```

Default Behavior:

- All notifications enabled by default
- Max 3 emails per week (configurable)
- No mute period

User Control:

- Toggle each notification type separately for email/in-app
- Adjust email frequency limit (1-10 per week)
- Temporarily mute all notifications (vacation mode)

3. ElementNote

Purpose: Store personal notes and details about collection items

```
model ElementNote {
  id String @id @default(cuid())
  userId String
  collectionItemId String @unique // 1:1 relationship

  // Personal details
  acquisitionDate DateTime?
  source String? // "Luciteria.com", "eBay", "Gift"
  pricePaid Decimal? @db.Decimal(10, 2)
  condition Condition?
  storageLocation String?
  notes String? @db.Text

  // Privacy
  isPrivate Boolean @default(true)

  // Metadata
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  user User @relation(fields: [userId], references: [id])
  collectionItem CollectionItem @relation(fields: [collectionItemId],
  references: [id])

  @@index([userId])
}

enum Condition {
  MINT // Perfect condition
  GOOD // Minor wear
  FAIR // Visible wear
  POOR // Significant damage
  UNKNOWN // Not assessed
}
```

Key Features:

- 1:1 relationship with CollectionItem (one note per item)
- All fields optional except `isPrivate`
- `pricePaid` uses Decimal for currency accuracy
- `notes` field is Text type (large content)

Privacy:

- `isPrivate = true` by default
- Phase 3+ may allow sharing notes with community

4. StockAlert (Optional - for future use)

Purpose: Track stock alerts for out-of-stock wishlist items

```

model StockAlert {
  id String @id @default(cuid())
  userId String
  elementSymbol String
  shopifyProductId String

  alertSent Boolean @default(false)
  alertSentAt DateTime?

  createdAt DateTime @default(now())

  @@unique([userId, shopifyProductId])
  @@index([shopifyProductId, alertSent])
}

```

Use Case:

- User adds out-of-stock item to wishlist
- System creates StockAlert record
- When product restocks, system:
 1. Finds all StockAlert records for that productId
 2. Queues WISHLIST_RESTOCK notifications
 3. Marks alertSent = true

5. AnalyticsSnapshot (Optional - for admin dashboard caching)

Purpose: Cache expensive aggregate queries for admin dashboard

```

model AnalyticsSnapshot {
  id String @id @default(cuid())
  snapshotType String // 'daily_stats', 'weekly_trends', 'element_popularity'
  data Json // Cached query results

  createdAt DateTime @default(now())
  expiresAt DateTime

  @@index([snapshotType, createdAt])
}

```

Use Case:

- Admin dashboard queries (top 10 most collected, etc.) are expensive
- Snapshot stores results for 5-minute cache
- Dashboard checks snapshot first, falls back to live query if expired

Example data:

```
{
  "totalUsers": 247,
  "totalCollections": 18943,
  "avgElementsPerUser": 76.7,
  "activeUsers7d": 89,
  "topCollected": [
    { "symbol": "C", "count": 198, "percentage": 80.2 },
    { "symbol": "Cu", "count": 187, "percentage": 75.7 }
  ]
}
```

Updated Models Summary

Models Modified (Phase 2)

1. **User**: Added relations for elementNotes, notifications, notificationPrefs
2. **CollectionItem**: Added contextLabel, contextOverridden, contextReason fields; added elementNote relation

Models Added (Phase 2)

1. **NotificationQueue**: Notification delivery system
2. **UserNotificationPreferences**: Per-user notification settings
3. **ElementNote**: Personal notes for collection items
4. **StockAlert**: (Optional) Stock restock tracking
5. **AnalyticsSnapshot**: (Optional) Admin dashboard caching

Enums Added (Phase 2)

1. **WishlistContext**: CORE, EXPLORATION, ASPIRATIONAL, UPGRADE_TARGET
 2. **NotificationType**: 5 notification types
 3. **NotificationStatus**: PENDING, SENT, FAILED, CANCELLED, READ
 4. **Condition**: MINT, GOOD, FAIR, POOR, UNKNOWN
-

Migration Strategy

Step 1: Add New Enums

```
// schema.prisma
enum WishlistContext {
  CORE
  EXPLORATION
  ASPIRATIONAL
  UPGRADE_TARGET
}

enum NotificationType {
  COMPLETION_UNLOCK
  NEAR_COMPLETION
  WISHLIST_RESTOCK
  HIGH_RELEVANCE_ARRIVAL
  MILESTONE_CELEBRATION
}

enum NotificationStatus {
  PENDING
  SENT
  FAILED
  CANCELLED
  READ
}

enum Condition {
  MINT
  GOOD
  FAIR
  POOR
  UNKNOWN
}
```

Step 2: Add Fields to CollectionItem

```
model CollectionItem {
  // ... existing fields ...

  contextLabel      WishlistContext?
  contextOverridden Boolean           @default(false)
  contextReason     String?

  elementNote       ElementNote?
}
```

Migration Command:

```
npx prisma migrate dev --name add_wishlist_context
```

Step 3: Create New Models

```
model NotificationQueue { ... }
model UserNotificationPreferences { ... }
model ElementNote { ... }
model StockAlert { ... }
model AnalyticsSnapshot { ... }
```

Migration Command:

```
npx prisma migrate dev --name add_phase2_models
```

Step 4: Seed Default Preferences

```
// prisma/seed.js
async function seedNotificationPreferences() {
  const users = await prisma.user.findMany();

  for (const user of users) {
    await prisma.userNotificationPreferences.upsert({
      where: { userId: user.id },
      update: {},
      create: {
        userId: user.id,
        emailCompletionUnlock: true,
        emailNearCompletion: true,
        emailWishlistRestock: true,
        emailHighRelevance: true,
        emailMilestone: true,
        inAppCompletionUnlock: true,
        inAppNearCompletion: true,
        inAppWishlistRestock: true,
        inAppHighRelevance: true,
        inAppMilestone: true,
        emailFrequencyLimit: 3
      }
    });
  }
}
```

Run Seed:

```
npx prisma db seed
```

Querying Patterns

1. Get User's Wishlist with Context

```
const wishlist = await prisma.collectionItem.findMany({
  where: {
    userId: userId,
    state: 'WISHLIST'
  },
  include: {
    element: true,
    elementNote: true
  },
  orderBy: [
    { contextLabel: 'asc' },
    { priority: 'desc' }
  ]
});

// Group by context
const grouped = {
  CORE: wishlist.filter(item => item.contextLabel === 'CORE'),
  EXPLORATION: wishlist.filter(item => item.contextLabel === 'EXPLORATION'),
  ASPIRATIONAL: wishlist.filter(item => item.contextLabel === 'ASPIRATIONAL'),
  UPGRADE_TARGET: wishlist.filter(item => item.contextLabel === 'UPGRADE_TARGET')
};
```

2. Queue Notification

```
await prisma.notificationQueue.create({
  data: {
    userId: userId,
    type: 'WISHLIST_RESTOCK',
    status: 'PENDING',
    relatedElementId: 'Ga',
    relatedProductId: 'shopify_123456',
    metadata: {
      elementSymbol: 'Ga',
      elementName: 'Gallium',
      format: '25.4mm',
      price: 49.99
    },
    deliveryMethod: ['in_app', 'email'],
    scheduledFor: new Date()
  }
});
```

3. Get Pending Notifications (Background Processor)

```
const pending = await prisma.notificationQueue.findMany({
  where: {
    status: 'PENDING',
    scheduledFor: {
      lte: new Date()
    }
  },
  include: {
    user: true
  },
  orderBy: {
    createdAt: 'asc'
  },
  take: 100
});
```

4. Get Element Notes

```
const note = await prisma.elementNote.findUnique({
  where: {
    collectionItemId: collectionItemId
  }
});
```

5. Admin: Top 10 Most Collected Elements

```
const topCollected = await prisma.collectionItem.groupBy({
  by: ['elementSymbol'],
  where: {
    state: 'OWNED'
  },
  _count: {
    id: true
  },
  orderBy: {
    _count: {
      id: 'desc'
    }
  },
  take: 10
});

// Enrich with element details
const enriched = await Promise.all(
  topCollected.map(async (item) => {
    const element = await prisma.element.findUnique({
      where: { symbol: item.elementSymbol }
    });
    return {
      symbol: item.elementSymbol,
      name: element.name,
      count: item._count.id,
      percentage: (item._count.id / totalUsers) * 100
    };
  })
);
```

6. Admin: Context Distribution

```
const contextDistribution = await prisma.collectionItem.groupBy({
  by: ['contextLabel'],
  where: {
    state: 'WISHLIST',
    contextLabel: {
      not: null
    }
  },
  _count: {
    id: true
  }
});

// Result:
// [
//   { contextLabel: 'CORE', _count: { id: 1876 } },
//   { contextLabel: 'EXPLORATION', _count: { id: 1542 } },
//   { contextLabel: 'ASPIRATIONAL', _count: { id: 789 } },
//   { contextLabel: 'UPGRADE_TARGET', _count: { id: 234 } }
// ]
```

Indexes & Performance

Critical Indexes

```
// CollectionItem indexes
@@index([userId, state]) // Fast user wishlist queries
@@index([elementSymbol]) // Fast element lookup
@@index([contextLabel]) // Fast context filtering

// NotificationQueue indexes
@@index([userId, status]) // User notification history
@@index([status, scheduledFor]) // Background processor query
@@index([type, createdAt]) // Analytics by type

// ElementNote indexes
@@index([userId]) // User's notes list

// StockAlert indexes
@@index([shopifyProductId, alertSent]) // Restock detection
```

Query Optimization Tips

1. **Batch Operations:** Use `createMany()` for bulk inserts
2. **Selective Includes:** Only include relations when needed
3. **Pagination:** Use `skip` and `take` for large result sets
4. **Aggregate Caching:** Use `AnalyticsSnapshot` for expensive admin queries
5. **Connection Pooling:** Configure Prisma connection pool for high concurrency

Data Validation

At Application Level (Prisma Schema)

- `@unique` constraints on email, collectionItemId
- `@default` values for booleans, timestamps
- Enum validation for contextLabel, condition, notification types

At API Level (Zod/Validation)

```
// Example: Element Note validation
const ElementNoteSchema = z.object({
  acquisitionDate: z.date().max(new Date()).optional(),
  source: z.string().max(100).optional(),
  pricePaid: z.number().min(0.01).max(999999.99).optional(),
  condition: z.enum(['MINT', 'GOOD', 'FAIR', 'POOR', 'UNKNOWN']).optional(),
  storageLocation: z.string().max(100).optional(),
  notes: z.string().max(1000).optional(),
  isPrivate: z.boolean().default(true)
});
```

Backup & Recovery

Daily Backups

- Automated PostgreSQL backups via Shopify hosting or AWS RDS
- Retain 30 days of backups

Critical Tables (Priority for Recovery)

1. User (account data)
2. CollectionItem (collection state)
3. ElementNote (personal data)
4. NotificationQueue (can be regenerated if lost)

Testing Migrations

```
# Create backup before migration
pg_dump -U user -d luciteria_db > backup_before_phase2.sql

# Run migration
npx prisma migrate dev

# If rollback needed
psql -U user -d luciteria_db < backup_before_phase2.sql
```

Security Considerations

Row-Level Security (RLS)

- Implement at API level: users can only access their own data

- Admin routes check `User.isStaff = true`

Sensitive Fields

- `pricePaid` : Private by default (not exposed in public APIs)
- `email` : Only visible to user and staff
- `notes` : Respects `isPrivate` flag

Audit Logging

- Track all admin actions (view user, export data)
 - Log notification delivery status
 - Track note modifications (via `updatedAt`)
-

Complete Schema (Full Prisma File)

```
// schema.prisma

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

generator client {
  provider = "prisma-client-js"
}

// =====
// PHASE 1 MODELS
// =====

model User {
  id          String      @id @default(cuid())
  email       String      @unique
  firstName   String?
  lastName    String?
  shopifyCustomerId String? @unique
  isStaff     Boolean      @default(false)
  preferredFormat String?

  createdAt      DateTime @default(now())
  updatedAt      DateTime @updatedAt

  collectionItems CollectionItem[]
  goals            Goal[]
  milestones       Milestone[]
  elementNotes     ElementNote[]
  notifications    NotificationQueue[]
  notificationPrefs UserNotificationPreferences?
}

model Element {
  id          String      @id @default(cuid())
  symbol      String      @unique
  name        String
  atomicNumber Int         @unique
  category    String
  group       Int?
  period      Int

  collectionItems CollectionItem[]

  @@index([category])
}

model CollectionItem {
  id          String      @id @default(cuid())
  userId      String
  elementSymbol String
  state       CollectionItemState
  format      String?

  contextLabel    WishlistContext?
  contextOverridden Boolean          @default(false)
  contextReason   String?

  priority        Int?          @default(0)
  shopifyProductId String?
}
```

```

    createdAt      DateTime      @default(now())
    updatedAt      DateTime      @updatedAt

    user           User           @relation(fields: [userId], references: [id])
    element         Element       @relation(fields: [elementSymbol],
references: [symbol])
    elementNote     ElementNote?

    @@unique([userId, elementSymbol, format])
    @@index([userId, state])
    @@index([elementSymbol])
    @@index([contextLabel])
}

model Goal {
  id              String          @id @default(cuid())
  userId          String
  title           String
  targetCount     Int
  completed       Boolean         @default(false)
  createdAt       DateTime       @default(now())

  user            User            @relation(fields: [userId], references: [id])
}

model Milestone {
  id              String          @id @default(cuid())
  userId          String
  name            String
  threshold       Int
  achieved        Boolean         @default(false)
  achievedAt      DateTime?

  user            User            @relation(fields: [userId], references: [id])

  @@unique([userId, name])
}

// =====
// PHASE 2 MODELS
// =====

model NotificationQueue {
  id              String          @id @default(cuid())
  userId          String
  type            NotificationType
  status          NotificationStatus @default(PENDING)

  relatedElementId String?
  relatedProductId String?
  relatedMilestone String?

  metadata        Json?
  relevanceScore  Int?

  deliveryMethod  String[]
  scheduledFor    DateTime?
  sentAt          DateTime?
  readAt          DateTime?
  clickedAt       DateTime?

  errorMessage    String?
  retryCount       Int            @default(0)
}

```

```

    createdAt      DateTime      @default(now())
    updatedAt      DateTime      @updatedAt

    user           User           @relation(fields: [userId], references: [id])

    @@index([userId, status])
    @@index([status, scheduledFor])
    @@index([type, createdAt])
}

model UserNotificationPreferences {
  id              String          @id @default(cuid())
  userId          String          @unique

  emailCompletionUnlock Boolean @default(true)
  emailNearCompletion Boolean @default(true)
  emailWishlistRestock Boolean @default(true)
  emailHighRelevance Boolean @default(true)
  emailMilestone Boolean @default(true)

  inAppCompletionUnlock Boolean @default(true)
  inAppNearCompletion Boolean @default(true)
  inAppWishlistRestock Boolean @default(true)
  inAppHighRelevance Boolean @default(true)
  inAppMilestone Boolean @default(true)

  emailFrequencyLimit Int @default(3)
  mutedUntil DateTime?

  createdAt      DateTime @default(now())
  updatedAt      DateTime @updatedAt

  user           User      @relation(fields: [userId], references: [id])
}

model ElementNote {
  id              String          @id @default(cuid())
  userId          String
  collectionItemId String          @unique

  acquisitionDate DateTime?
  source          String?
  pricePaid       Decimal? @db.Decimal(10, 2)
  condition       Condition?
  storageLocation String?
  notes           String? @db.Text

  isPrivate       Boolean @default(true)

  createdAt      DateTime @default(now())
  updatedAt      DateTime @updatedAt

  user           User      @relation(fields: [userId], references: [id])
  collectionItem CollectionItem @relation(fields: [collectionItemId],
  references: [id])

  @@index([userId])
}

model StockAlert {
  id              String          @id @default(cuid())
  userId          String

```

```

elementSymbol      String
shopifyProductId   String

alertSent          Boolean @default(false)
alertSentAt        DateTime?

createdAt          DateTime @default(now())

@@unique([userId, shopifyProductId])
@@index([shopifyProductId, alertSent])
}

model AnalyticsSnapshot {
  id                String @id @default(cuid())
  snapshotType      String
  data              Json

  createdAt         DateTime @default(now())
  expiresAt         DateTime

  @@index([snapshotType, createdAt])
}

// =====
// ENUMS
// =====

enum CollectionItemState {
  OWNED
  WISHLIST
  WATCHLIST
  MISSING
}

enum WishlistContext {
  CORE
  EXPLORATION
  ASPIRATIONAL
  UPGRADE_TARGET
}

enum NotificationType {
  COMPLETION_UNLOCK
  NEAR_COMPLETION
  WISHLIST_RESTOCK
  HIGH_RELEVANCE_ARRIVAL
  MILESTONE_CELEBRATION
}

enum NotificationStatus {
  PENDING
  SENT
  FAILED
  CANCELLED
  READ
}

enum Condition {
  MINT
  GOOD
  FAIR
  POOR
}

```



Summary

New Tables: 5 (NotificationQueue, UserNotificationPreferences, ElementNote, StockAlert, AnalyticsSnapshot)

Modified Tables: 2 (User, CollectionItem)

New Enums: 4 (WishlistContext, NotificationType, NotificationStatus, Condition)

Migration Complexity: Low-Medium

Estimated Migration Time: 10-15 minutes (including seed)

Philosophy: Extend existing schema with minimal disruption. All Phase 2 features are additive—no breaking changes to Phase 1 data.